

Ingeniería del Software 3

Clase 2 · Contenedores: Docker + Compose

UCC · Ingeniería en Sistemas · 2026 · Ing. Ariel Schwindt

Warm-up: ¿cómo va el TP1? + agenda de hoy

- Check TP1:
 - ¿Vas al día con la guía? La defensa del TP1 es en la clase 5, junto con el 2, el 3 y el 4

Bloque de hoy	Qué pasa
Teoría	El problema del entorno · contenedores vs VMs · imágenes, layers, Dockerfile, multi-stage · volúmenes, redes, compose
El TP2	La app del semestre 🌟 + tareas + entregables
La práctica	La guía del TP2, paso a paso — tiene su propio video

El problema: "en mi máquina funciona"

El software nunca viaja solo

- Runtime en otra versión
- Librería del sistema que falta
- Variable de entorno distinta
- Config que alguien tocó a mano
- El entorno es **IMPLÍCITO** y artesanal

La respuesta del contenedor

- App + entorno empaquetados JUNTOS
- Unidad inmutable y portable
- Idéntica en tu máquina, QA y prod
- El entorno queda **DECLARADO** en un archivo versionado (Dockerfile)

¿Te suena? "Si no está en el repo, no existe" — el entorno también entra al repo.

Contenedores vs máquinas virtuales

	Máquina Virtual	Contenedor
Qué virtualiza	Hardware completo (SO invitado propio)	Solo el proceso (comparte el kernel Linux — en Mac/Windows, el de una VM)
Peso típico	GBs	MBs
Arranque	Minutos	Segundos o menos
Densidad por host	Pocas	Decenas / cientos
Aislamiento	Fuerte (hipervisor)	De proceso (namespaces + cgroups)

- **El truco: un contenedor NO arranca un sistema operativo. Es un proceso común, al que el kernel de Linux le hace creer que está solo en la máquina (namespaces) y le pone un techo de CPU y memoria (cgroups)**
- **¿Y en Mac o Windows, que no tienen kernel de Linux? Docker Desktop levanta una máquina virtual con Linux adentro, y tus contenedores corren ahí**
- No son enemigos: en la nube tus contenedores corren ADENTRO de máquinas virtuales (la VM aísla clientes; el contenedor empaqueta apps)

El modelo mental: imagen, contenedor, registry

Pieza	Analogía	Qué es
Imagen	La clase	Paquete inmutable: app + dependencias + filesystem, en capas de solo lectura
Contenedor	La instancia	Una imagen en ejecución + una capa de escritura efímera
Registry	El GitHub de las imágenes	Donde se publican y descargan (Docker Hub, ghcr.io, ACR)

`ghcr.io/ariel/mi-api:v0.1.0` ← *registry / usuario / imagen : tag (semver – la disciplina del TP1 sigue acá)*

- Podés correr N contenedores de la misma imagen (como N objetos de una clase)
- **Todo lo que el contenedor escribe muere con él — el estado va en volúmenes (en 4 slides)**

Layers: por qué las imágenes se apilan

Capa de **ESCRITURA** del contenedor (efímera — muere con él)

Metadatos: ENTRYPOINT, ENV, EXPOSE (no son capa)

Capa: binarios de tu app (COPY)

Capa: dependencias (RUN restore/install)

Capa base: runtime (FROM ...)

- Crean capa RUN, COPY y ADD — el resto (WORKDIR, EXPOSE, ENV, ENTRYPOINT) sólo escribe metadatos
- Las capas se cachean y se COMPARTEN entre imágenes
- **Cambia una capa → se rehacen esa y todas las posteriores (arriba en la pila = después en el Dockerfile) — el orden importa: próximo slide**
- Por eso el 1er pull es lento y los siguientes casi instantáneos

↑ las 4 capas de abajo: SOLO LECTURA — compartidas y cacheadas entre imágenes

El Dockerfile: la receta (y por qué el orden importa)

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY ["MiApi.csproj", "."]      # ① primero SOLO el archivo de dependencias
RUN dotnet restore "MiApi.csproj" # ② capa cacheable: se rehace solo si cambia
COPY . .                        # ③ recién ahora el resto del código
RUN dotnet publish -c Release -o /app/publish

FROM mcr.microsoft.com/dotnet/aspnet:8.0 # ④ etapa final: sin compilador
WORKDIR /app
COPY --from=build /app/publish .
EXPOSE 8080                               # ⑤ DOCUMENTA el puerto (no lo publica)
ENTRYPOINT ["dotnet", "MiApi.dll"]        # ⑥ qué ejecuta el contenedor
# CMD ["--urls", "http://+:8080"]         # ⑦ argumentos por defecto (opcional)
```

- El Dockerfile es **CÓDIGO**: se versiona, se revisa en PRs, se audita
- Instrucciones clave: FROM (base) • COPY (archivos) • RUN (comandos de build) • EXPOSE (DOCUMENTA el puerto — no lo publica: eso lo hace -p / ports:)
- ENTRYPOINT = el ejecutable (se puede pisar con docker run --entrypoint) • CMD = argumentos por defecto, reemplazables desde docker run
- **Regla de oro #1: dependencias ANTES que código → cambiar una línea de código no re-descarga todo**

Multi-stage: el SDK no viaja a producción

```
# ---- Etapa 1: build (SDK completo) ----  
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build  
...restore, copy, publish... → /app/publish  
  
# ---- Etapa 2: runtime (mínima) ----  
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final  
COPY --from=build /app/publish .      # ← SOLO los binarios cruzan  
ENTRYPOINT ["dotnet", "MiApi.dll"]
```

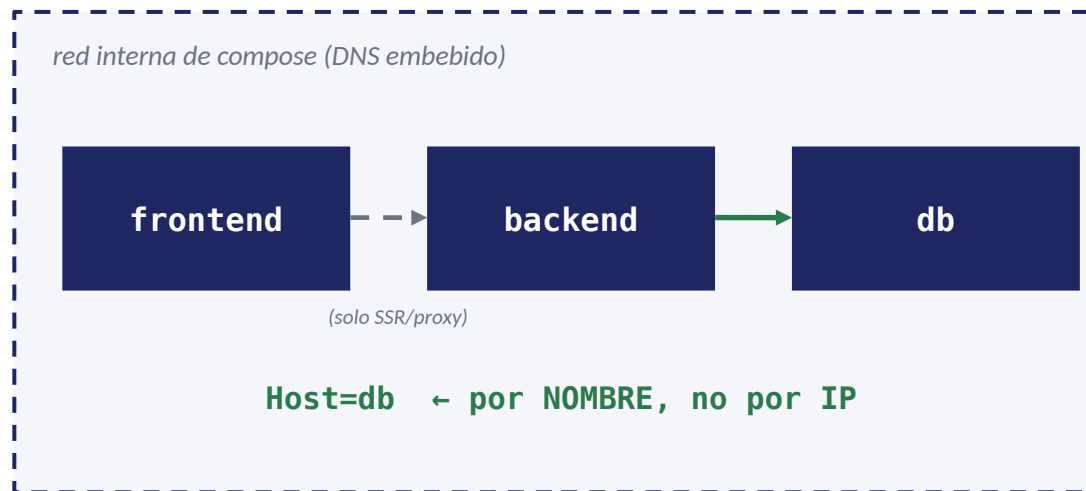
- **Compilar necesita el SDK; ejecutar solo necesita el runtime**
- Resultado: imagen final varias veces más chica, más rápida de mover, y con MENOS superficie de ataque (sin compilador ni herramientas adentro)
- **Regla de oro #2: multi-stage siempre — en esta materia es obligatorio**

Persistencia: el estado vive en volúmenes

- Los contenedores son EFÍMEROS por diseño:
 - para la app es una feature: descartables y reemplazables = deploys y rollbacks triviales
 - para los datos sería una catástrofe: ¿la BD pierde todo con cada recreación?
- La solución: separar contenedor (efímero) de estado (persistente)
 - un VOLUMEN es almacenamiento gestionado por Docker, montado dentro del contenedor, que SOBREVIVE a su destrucción

```
services:
  db:
    image: postgres:16-alpine
    volumes:
      - db_data:/var/lib/postgresql/data    # el contenedor muere; los datos NO
```

Redes: los servicios se encuentran por NOMBRE



- TRES ámbitos distintos:
 - contenedor→contenedor: por nombre de servicio (back→db)
 - host→contenedor: solo puertos publicados (ports:)
 - ⚠ La SPA es el caso trampa: su JavaScript corre en el navegador, que está FUERA de la red de compose. El nombre 'backend' ahí no existe.
 - La solución: el front pide a /api/... sin nombrar ningún host, y nginx —que sí está adentro— reenvía ese /api al backend.
 - Así la misma imagen sirve en cualquier entorno y no hace falta CORS: para el navegador, todo viene del mismo lugar.
 - La alternativa —URL completa en el código— anda, pero obliga a CORS y ata la imagen a ese entorno.

Desacople de topología: no importa en qué IP cayó el contenedor — el nombre del servicio siempre lo encuentra.

Compose: el sistema completo, declarado

```
services:
  db:      { image: postgres:16-alpine, volumes: [db_data:...], healthcheck: ... }
  backend: { build: ./backend, environment: [Host=db], depends_on: db }
  frontend: { build: ./frontend, ports: ["3000:80"], depends_on: backend }
volumes: { db_data }
```

- `docker compose up -d` → el sistema ENTERO, con un comando, en cualquier máquina
- **DECLARATIVO:** describís el estado deseado ('existen estos 3 servicios conectados así'), no los pasos
- Es tu primera infraestructura declarada en un archivo — la semilla conceptual del TP8 (IaC)
- Onboarding de un dev nuevo: `git clone` + `docker compose up`. Fin del documento de instalación de 20 páginas

"Arrancó" no es "está listo": depends_on + healthcheck

```
db:
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"] # ¿la BD ACEPTA conexiones?
    interval: 5s
  backend:
    depends_on:
      db:
        condition: service_healthy # espera a que esté LISTA, no solo iniciada
```

- **depends_on solo ordena el ARRANQUE — no garantiza que el servicio esté operativo**
- El contenedor de la BD 'arrancó' en 1 segundo; PostgreSQL tarda varios más en aceptar conexiones → el backend explota en el medio
- **'Arrancó' ≠ 'está listo' reaparece en TODO sistema distribuido que toque (y en el TP6 con los deploys)**

Secretos: el .env que NO se commitea

Archivo	¿Al repo?	Qué contiene
.env	NO — está en .gitignore	Los valores reales (passwords, tokens)
.env.example	SÍ	Las CLAVES esperadas con valores de ejemplo — documentación ejecutable

- El compose los referencia: `POSTGRES\_PASSWORD: \${DB\_PASSWORD}` — el YAML se commitea, el valor no
- **Un password commiteado en un repo público queda en el historial PARA SIEMPRE (borrarlo del archivo no lo borra de la historia)**
- En TP4 estos secretos migran a la plataforma de CI; en TP9 activamos el escáner que bloquea el push de un secreto
- **La disciplina empieza HOY: ningún valor sensible entra al repo, nunca**

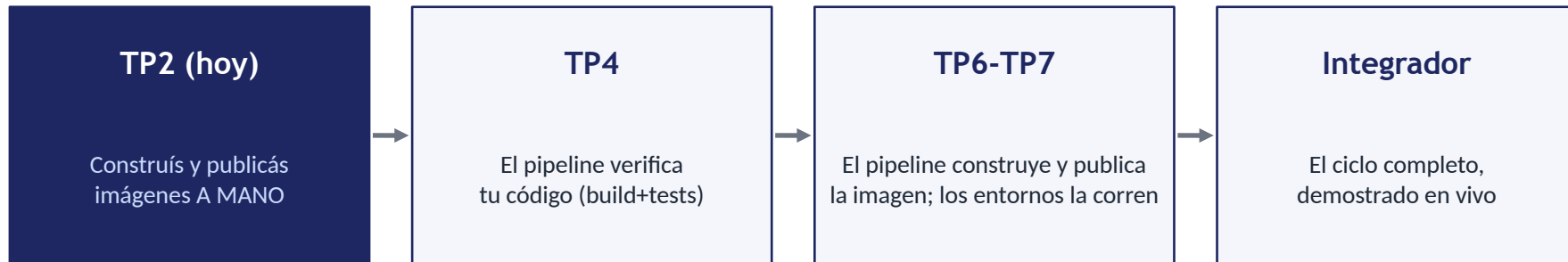
★ La decisión del día: tu app del semestre

- La app que elijas HOY te acompaña hasta el final: TP4 le arma CI, TP5 tests, TP6 deploy, TP7 e2e, TP8 IaC, TP9 seguridad... y es la base del INTEGRADOR

Requisito / criterio	Detalle
Backend con API	El lenguaje lo elegís vos (.NET, Node, Java, Python, Go...)
Frontend	SPA o SSR (React, Angular, Vue...)
Base de datos	Relacional o no
Corre local HOY	Probala ANTES de comprometerte — sin magia ni servicios externos raros
La entendés	El Integrador exige modificarla EN VIVO frente al profesor
¿Tests?	Tiene (o podés escribirle) — lo vas a necesitar en TP4 y TP5
Tamaño: CHICA	CRUD + 2-3 pantallas alcanza. Más grande NO suma nota — solo suma fricción

🔴 **INDIVIDUAL:** cada uno una app DISTINTA — dos personas no pueden entregar la misma. NO sirve el trabajo grupal de otra materia · Propia u open-source que entiendas · Se puede cambiar hasta el TP4 (rehaciendo TP2-3) — después, consultar.

El contenedor como unidad de entrega (hacia dónde va esto)



- La imagen taggeada con semver en un registry **ES** el artefacto de release
- Lo que se prueba en QA es EXACTAMENTE lo que corre en PROD (misma imagen, bit a bit) — se acabó el 'pero en QA andaba'
- Por eso hoy publican sus primeras imágenes en ghcr.io / Docker Hub: es el eslabón que el resto de la materia da por hecho

La práctica: la guía del TP2, paso a paso

- **La secuencia de la guía §3, sobre el SAMPLE de la cátedra (práctica) — hay un video que la recorre entera. Después repetís todo sobre TU app, que es lo que se entrega:**
 - 1. La app corriendo local SIN Docker (punto de partida honesto)
 - 2. Dockerfile multi-stage del backend → build → medir tamaño vs SDK
 - 3. Dockerfile del frontend (build + nginx)
 - 4. docker-compose.yml: BD con volumen + healthcheck, red, .env
 - 5. docker compose up -d → sistema completo → prueba de persistencia (down/up vs down -v)
 - 6. Publicar imagen en ghcr.io con tag semver
- Dos tropiezos que le pasan a todos: connection string a localhost • backend arrancando antes que la BD
- **Checkpoint tras checkpoint: si uno no te da, frená ahí y traé la duda**

TP2 — Contenedores: qué tenés que hacer

- **Escenario:** cualquiera que clone tu repo levanta el sistema completo con UN comando
- 1 • Elegir la app del semestre (contra los criterios del slide 🌟) y justificarla en decisiones.md
- 2 • Dockerfile MULTI-STAGE para backend y frontend, con .dockerignore
- 3 • docker-compose.yml con el sistema completo:
 - volumen para la BD • red (servicios por nombre) • depends_on + healthcheck • secretos vía .env (+ .env.example commiteado)
- 4 • Imágenes de back y front publicadas en registry gratuito (ghcr.io / Docker Hub) con tag semver y visibilidad pública
- Todo por PR sobre tu propio repo — las reglas del TP1 siguen vigentes

TP2 — Entregables, defensa y evaluación

Entregable	Contenido
URL del repo público	Dockerfiles + .dockerignore + nginx.conf + compose + docker-compose.registry.yml + .env.example + README de arranque + la app
decisiones.md	App elegida (vs criterios) · imágenes base y multi-stage · qué persiste · problemas · uso de IA
evidencias.md	compose up end-to-end · persistencia (down/up vs down -v) · tamaños · registry público

- **Defensa (en P1, clase 5): 25% técnico · 25% decisiones+evidencias · 50% defensa oral**
- Preguntas típicas: ¿imagen vs contenedor? ¿ENTRYPOINT vs CMD? ¿por qué multi-stage? ¿qué pasa con down -v? ¿qué es 'db' en tu connection string?
- **En vivo: clonar tu repo en una carpeta limpia y levantarlo siguiendo TU README (son dos pasos: cp .env.example .env, y compose up)**
- **Manos a la obra, en DOS pasadas: 1) hacé la guía completa sobre la app de la cátedra, para practicar · 2) cuando tengas elegida TU app, repetí todo sobre ella — eso es lo que se entrega**

Cierre — para la próxima clase

1

Elegí tu app ESTA SEMANA

Probala corriendo local antes de comprometerte — criterios en la guía §3.3

2

Arrancá el TP2: dockerizala

Checkpoint a checkpoint; los bugs de la demo (localhost, healthcheck) te van a pasar — ya sabés resolverlos

3

TP1 en marcha

Se defiende en la clase 5 junto con TP2, 3 y 4 — no acumules

4

Próxima clase: planificación y trazabilidad

GitHub Projects / Azure Boards — vas a planificar el resto del semestre sobre TU app